



Deep Learning

Lecture 1.2: Optimization Primer

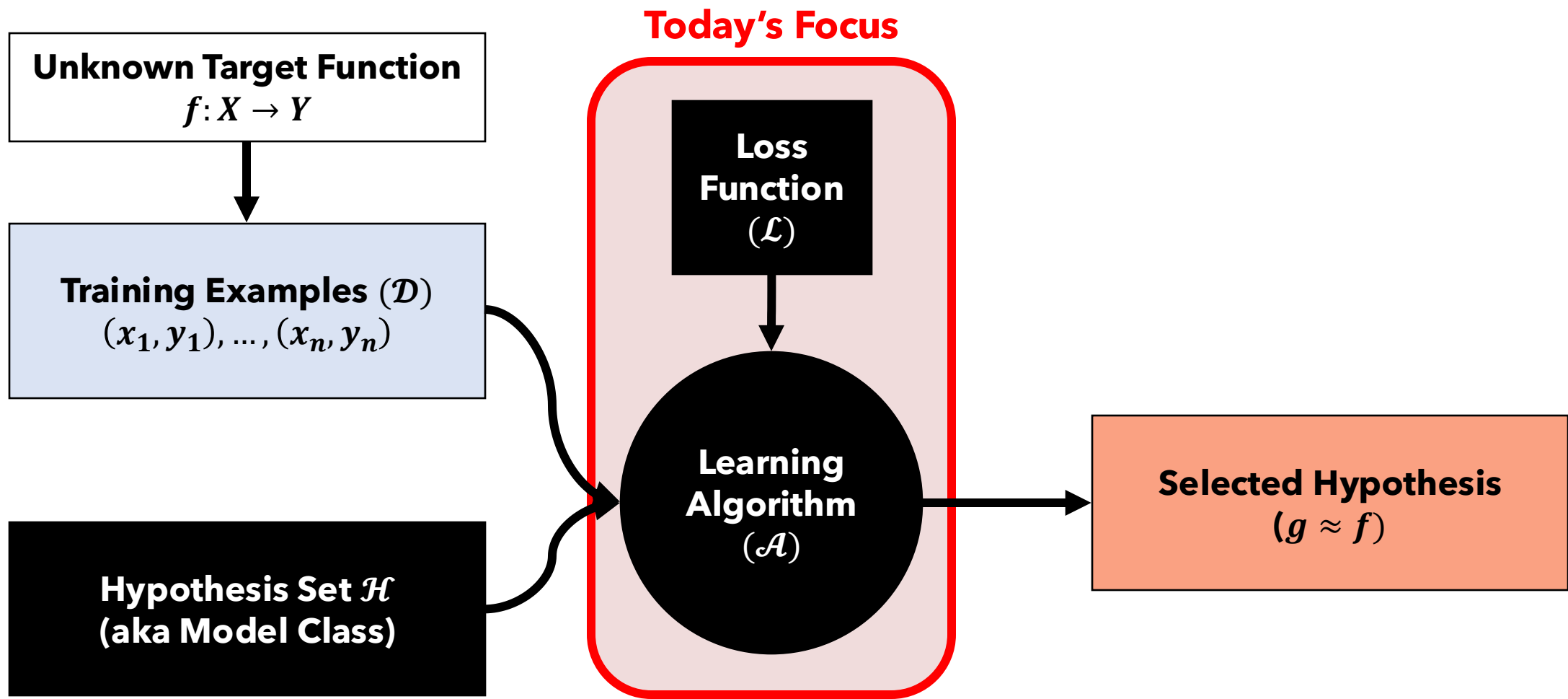


Today's Learning Objectives



Be able to answer:

- What is a loss function?
 - What are typical loss functions?
- What are gradients? Jacobians? Hessians?
- What is optimization?
 - What are first-order methods?
 - What is gradient descent?
 - When does gradient descent reach optima?
 - What makes a function convex / non-convex?
 - What is the intuition behind second order methods?
 - Why are they typically not used in deep learning?



Observations of some phenomenon



Designed by us



Result of our learning



Loss Function - A function measuring “how bad” a model’s output is, usually relative to some gold-standard for what the output should be.

- Hard question – how will you define “bad vs good” for your problem?
- Differentiable loss functions are easier to work with in deep learning.



Suppose your goal is to learn a function $f(x; \theta)$ with parameters θ that outputs a score corresponding to the class of the input $y \in \{-1, 1\}$

	$f(x) < 0$	$f(x) \geq 0$
$y = 1$	Incorrect	Correct
$y = -1$	Correct	Incorrect

What you probably really want is to be right as often as possible on new data:

$$\min_{\theta} E_{x,y} [I[yf(x) \leq 0]]$$

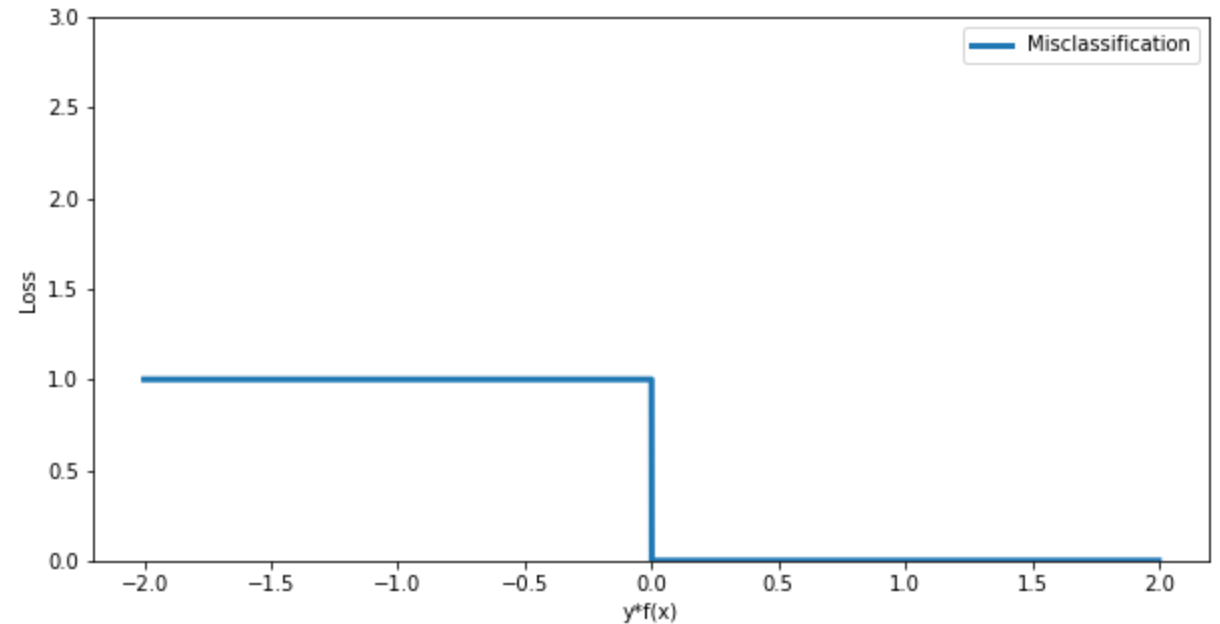
$$I[y * f(x) < 0] = \begin{cases} 0, & yf(x) > 0 \\ 1 & yf(x) \leq 0 \end{cases}$$

Call the classifier that minimizes this the Bayes classifier which achieves the minimal expected loss called the Bayes error.



Call this the misclassification loss:

$$L_{\text{misclass}} = I[yf(x) \leq 0]$$



Hard to deal with this loss directly if you are relying on gradients.

Idea: Pick a loss that upper-bounds this one, but is easier to work with!



Misclassification:

$$L_{\text{misclass}} = I[yf(x) < 0]$$

L1 / L2:

$$L_1 = |yf(x; \theta) - 1|$$

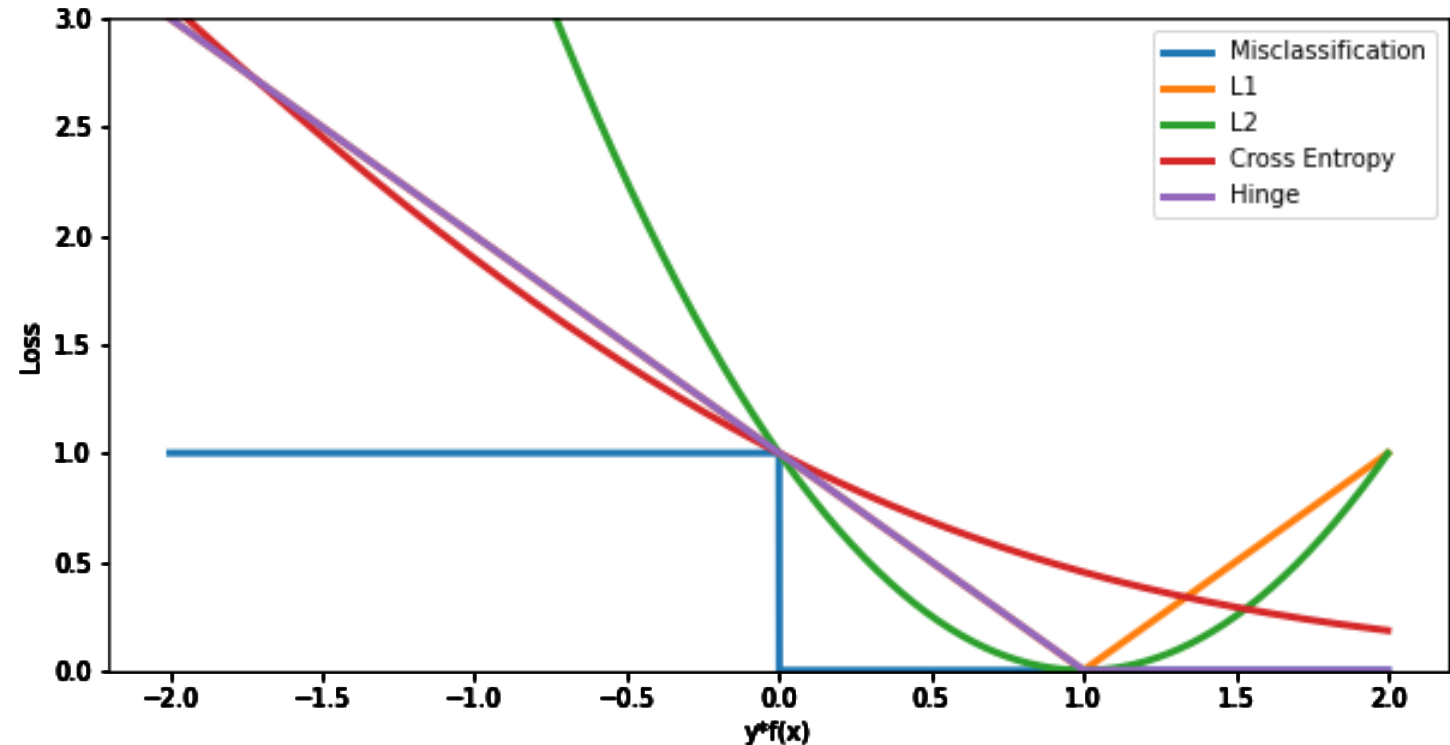
$$L_2 = (yf(x; \theta) - 1)^2$$

Cross Entropy:

$$L_{CE} = \log_2(1 + e^{-yf(x)})$$

Hinge:

$$L_{\text{Hinge}} = \max(0, 1 - yf(x))$$





Suppose your goal is to learn a function $f(x; \theta)$ with parameters θ that outputs a continuous valued y

L2 - Squared Error

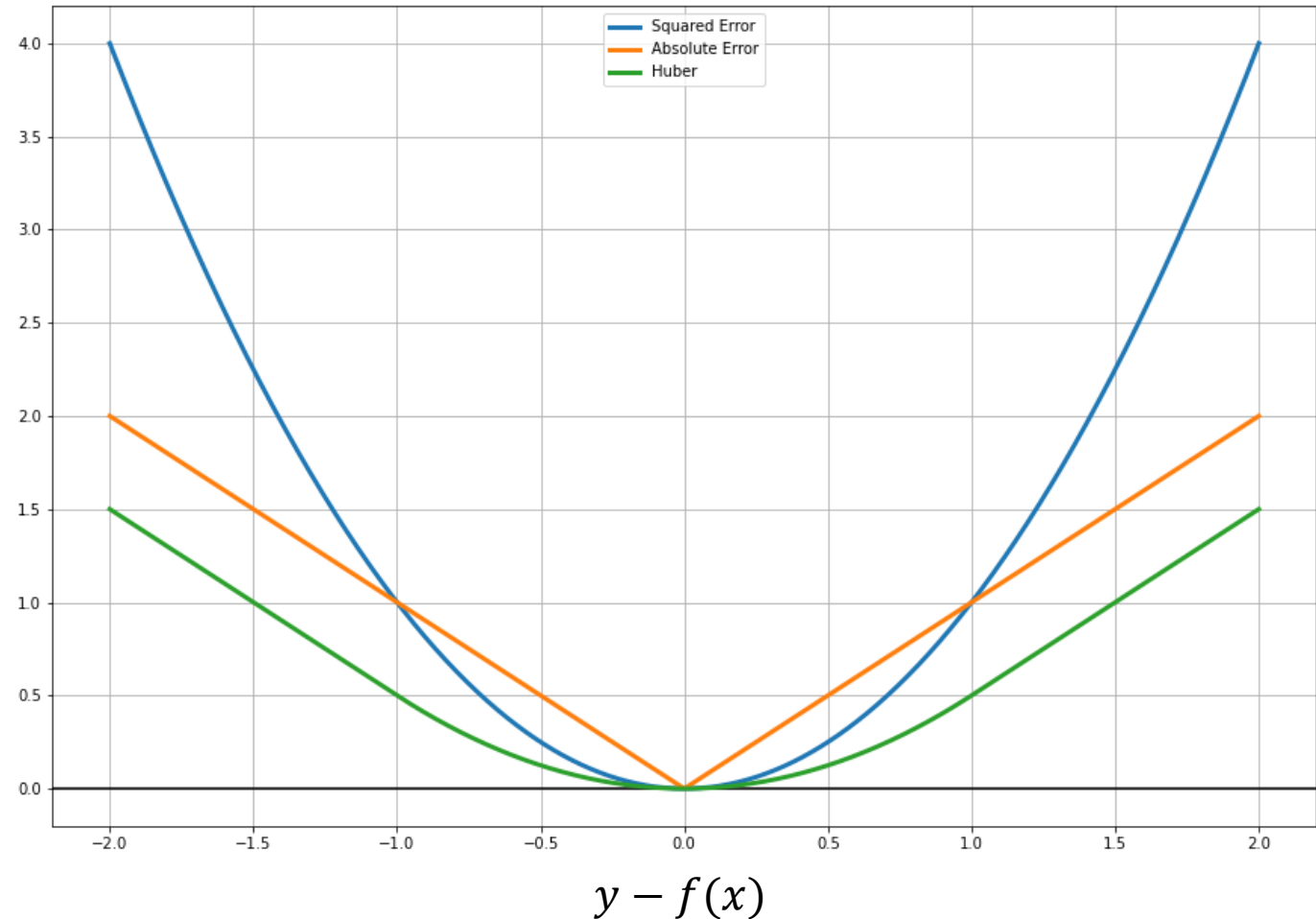
$$L(y, f(x)) = (y - f(x))^2$$

L1- Absolute Error

$$L(y, f(x)) = |y - f(x)|$$

Huber

$$L(y, f(x)) = \begin{cases} \frac{1}{2}(y - f(x))^2 & \text{if } |y - f(x)| < \delta \\ \delta|y - f(x)| - \frac{1}{2}\delta^2 & \text{else} \end{cases}$$



Today's Learning Objectives

Be able to answer:

- ~~What is a loss function?~~
- ~~What are typical loss functions?~~
- What are gradients? Jacobians? Hessians?
- What is optimization?
 - What are first-order methods?
 - What is gradient descent?
 - When does gradient descent reach optima?
 - What makes a function convex / non-convex?
 - What is the intuition behind second order methods?
 - Why are they typically not used in deep learning?





Derivative of f with respect to x :

$$\frac{d}{dx} f(x) = \lim_{h \rightarrow 0} \frac{f(x + h) - f(x)}{h}$$

Partial Derivatives of f with respect to x and y :

$$\frac{\partial}{\partial x} f(x, y) = \lim_{h \rightarrow 0} \frac{f(x + h, y) - f(x, y)}{h}$$

$$\frac{\partial}{\partial y} f(x, y) = \lim_{h \rightarrow 0} \frac{f(x, y + h) - f(x, y)}{h}$$



Scalar

1

$x \in \mathbb{R}$

Vector

$\begin{bmatrix} 1 \\ 2 \end{bmatrix}$

$x \in \mathbb{R}^2$

Matrix

$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

$x \in \mathbb{R}^{2 \times 2}$

Tensor

$\begin{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} & \begin{bmatrix} 3 & 2 \end{bmatrix} \\ \begin{bmatrix} 1 & 7 \end{bmatrix} & \begin{bmatrix} 5 & 4 \end{bmatrix} \end{bmatrix}$

$x \in \mathbb{R}^{2 \times 2 \times 2}$



Tensors can have more than three dimensions:

Vector

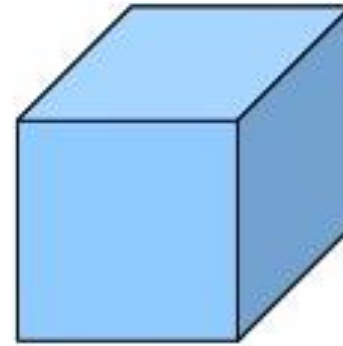


1d-tensor

Matrix



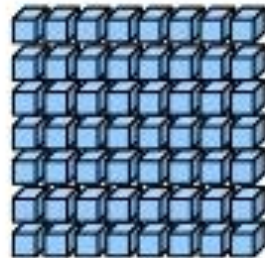
2d-tensor



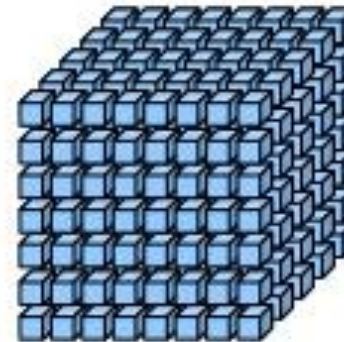
3d-tensor



4d-tensor



5d-tensor



6d-tensor



Scalars: $y, x \in \mathbb{R}$

Vectors: $\mathbf{y} \in \mathbb{R}^d, \mathbf{x} \in \mathbb{R}^c$

Matrices: $\mathbf{X} \in \mathbb{R}^{m \times n}, \mathbf{Y} \in \mathbb{R}^{o \times p}$

Our convention: Numerator dim $\rightarrow$ rows, Denominator dim $\rightarrow$ cols

$\frac{\delta \text{vector}}{\delta \text{scalar}}$

$$\frac{\delta \mathbf{y}}{\delta x} = \begin{bmatrix} \frac{\delta y_1}{\delta x} \\ \frac{\delta y_2}{\delta x} \\ \vdots \\ \frac{\delta y_d}{\delta x} \end{bmatrix}_{d \times 1}$$

$\frac{\delta \text{scalar}}{\delta \text{vector}}$

$$\frac{\delta y}{\delta \mathbf{x}} = \begin{bmatrix} \frac{\delta y}{\delta x_1} & \frac{\delta y}{\delta x_2} & \cdots & \frac{\delta y}{\delta x_c} \end{bmatrix}_{1 \times c}$$



Scalars: $y, x \in \mathbb{R}$

Vectors: $\mathbf{y} \in \mathbb{R}^d, \mathbf{x} \in \mathbb{R}^c$

Matrices: $\mathbf{X} \in \mathbb{R}^{m \times n}, \mathbf{Y} \in \mathbb{R}^{o \times p}$

Our convention: Numerator dim $\rightarrow$ rows, Denominator dim $\rightarrow$ cols

$$\frac{\delta \mathbf{y}}{\delta x} = \begin{matrix} \frac{\delta \text{vector}}{\delta \text{scalar}} \\ \left[\begin{array}{c} \frac{\delta y_1}{\delta x} \\ \frac{\delta y_2}{\delta x} \\ \vdots \\ \frac{\delta y_d}{\delta x} \end{array} \right]_{d \times 1} \end{matrix}$$

$$\mathbf{y}(x) = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} -5x \\ x^2 \end{bmatrix}_{2 \times 1}$$



Scalars: $y, x \in \mathbb{R}$

Vectors: $\mathbf{y} \in \mathbb{R}^d, \mathbf{x} \in \mathbb{R}^c$

Matrices: $\mathbf{X} \in \mathbb{R}^{m \times n}, \mathbf{Y} \in \mathbb{R}^{o \times p}$

Our convention: Numerator dim $\rightarrow$ rows, Denominator dim $\rightarrow$ cols

$$\frac{\delta \mathbf{y}}{\delta x} = \begin{matrix} \frac{\delta \text{vector}}{\delta \text{scalar}} \\ \left[\begin{array}{c} \frac{\delta y_1}{\delta x} \\ \frac{\delta y_2}{\delta x} \\ \vdots \\ \frac{\delta y_d}{\delta x} \end{array} \right]_{d \times 1} \end{matrix}$$

$$\mathbf{y}(x) = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} -5x \\ x^2 \end{bmatrix}_{2 \times 1}$$

$$\frac{\delta \mathbf{y}}{\delta x} = \begin{bmatrix} -5 \\ 2x \end{bmatrix}_{2 \times 1}$$



Scalars: $y, x \in \mathbb{R}$

Vectors: $\mathbf{y} \in \mathbb{R}^d, \mathbf{x} \in \mathbb{R}^c$

Matrices: $\mathbf{X} \in \mathbb{R}^{m \times n}, \mathbf{Y} \in \mathbb{R}^{o \times p}$

Our convention: Numerator dim $\rightarrow$ rows, Denominator dim $\rightarrow$ cols

$\frac{\delta \text{scalar}}{\delta \text{vector}}$

$$\frac{\delta y}{\delta \mathbf{x}} = \begin{bmatrix} \frac{\delta y}{\delta x_1} & \frac{\delta y}{\delta x_2} & \cdots & \frac{\delta y}{\delta x_c} \end{bmatrix}_{1 \times c}$$

$$y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} = \sum_i w_i x_i \quad \mathbf{w} \in \mathbb{R}^c$$



Scalars: $y, x \in \mathbb{R}$

Vectors: $\mathbf{y} \in \mathbb{R}^d, \mathbf{x} \in \mathbb{R}^c$

Matrices: $\mathbf{X} \in \mathbb{R}^{m \times n}, \mathbf{Y} \in \mathbb{R}^{o \times p}$

Our convention: Numerator dim $\rightarrow$ rows, Denominator dim $\rightarrow$ cols

$\frac{\delta \text{scalar}}{\delta \text{vector}}$

$$\frac{\delta y}{\delta \mathbf{x}} = \begin{bmatrix} \frac{\delta y}{\delta x_1} & \frac{\delta y}{\delta x_2} & \cdots & \frac{\delta y}{\delta x_c} \end{bmatrix}_{1 \times c}$$

$$y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} = \sum_i w_i x_i \quad \mathbf{w} \in \mathbb{R}^c$$

$$\begin{aligned} \frac{\delta y}{\delta \mathbf{x}} &= [w_1 \quad w_2 \quad \cdots \quad w_c]_{1 \times c} \\ &= \mathbf{w}^T \end{aligned}$$



Scalars: $y, x \in \mathbb{R}$

Vectors: $\mathbf{y} \in \mathbb{R}^d, \mathbf{x} \in \mathbb{R}^c$

Matrices: $\mathbf{X} \in \mathbb{R}^{m \times n}, \mathbf{Y} \in \mathbb{R}^{o \times p}$

Our convention: Numerator dim $\rightarrow$ rows, Denominator dim $\rightarrow$ cols

$\frac{\delta \text{vector}}{\delta \text{vector}}$

$$\frac{\delta \vec{\mathbf{y}}}{\delta \vec{\mathbf{x}}} = \begin{bmatrix} \frac{\delta y_1}{\delta x_1} & \frac{\delta y_1}{\delta x_2} & \cdots & \frac{\delta y_1}{\delta x_c} \\ \frac{\delta y_2}{\delta x_1} & \frac{\delta y_2}{\delta x_2} & \cdots & \frac{\delta y_2}{\delta x_c} \\ \vdots & \vdots & \cdots & \vdots \\ \frac{\delta y_d}{\delta x_1} & \frac{\delta y_d}{\delta x_2} & \cdots & \frac{\delta y_d}{\delta x_c} \end{bmatrix}_{d \times c}$$

Called the Jacobian



Scalars: $y, x \in \mathbb{R}$

Vectors: $\mathbf{y} \in \mathbb{R}^d, \mathbf{x} \in \mathbb{R}^c$

Matrices: $\mathbf{X} \in \mathbb{R}^{m \times n}, \mathbf{Y} \in \mathbb{R}^{o \times p}$

Our convention: Numerator dim $\rightarrow$ rows, Denominator dim $\rightarrow$ cols

$$\frac{\delta \mathbf{y}}{\delta \mathbf{x}} = \begin{bmatrix} \frac{\delta y_1}{\delta x_1} & \frac{\delta y_1}{\delta x_2} & \cdots & \frac{\delta y_1}{\delta x_c} \\ \frac{\delta y_2}{\delta x_1} & \frac{\delta y_2}{\delta x_2} & \cdots & \frac{\delta y_2}{\delta x_c} \\ \vdots & \vdots & \cdots & \vdots \\ \frac{\delta y_d}{\delta x_1} & \frac{\delta y_d}{\delta x_2} & \cdots & \frac{\delta y_d}{\delta x_c} \end{bmatrix}_{d \times c}$$

Called the Jacobian

$$\mathbf{y}(\mathbf{x}) = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} -5x_1 + x_2 \\ x_1^2 \end{bmatrix}$$



Scalars: $y, x \in \mathbb{R}$

Vectors: $\mathbf{y} \in \mathbb{R}^d, \mathbf{x} \in \mathbb{R}^c$

Matrices: $\mathbf{X} \in \mathbb{R}^{m \times n}, \mathbf{Y} \in \mathbb{R}^{o \times p}$

Our convention: Numerator dim $\rightarrow$ rows, Denominator dim $\rightarrow$ cols

$$\frac{\delta \mathbf{y}}{\delta \mathbf{x}} = \begin{bmatrix} \frac{\delta y_1}{\delta x_1} & \frac{\delta y_1}{\delta x_2} & \cdots & \frac{\delta y_1}{\delta x_c} \\ \frac{\delta y_2}{\delta x_1} & \frac{\delta y_2}{\delta x_2} & \cdots & \frac{\delta y_2}{\delta x_c} \\ \vdots & \vdots & \cdots & \vdots \\ \frac{\delta y_d}{\delta x_1} & \frac{\delta y_d}{\delta x_2} & \cdots & \frac{\delta y_d}{\delta x_c} \end{bmatrix}_{d \times c}$$

Called the Jacobian

$$\mathbf{y}(\mathbf{x}) = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} -5x_1 + x_2 \\ x_1^2 \end{bmatrix}$$

$$\frac{\delta \mathbf{y}}{\delta \mathbf{x}} = \begin{bmatrix} \frac{\delta y_1}{\delta x_1} & \frac{\delta y_1}{\delta x_2} \\ \frac{\delta y_2}{\delta x_1} & \frac{\delta y_2}{\delta x_2} \end{bmatrix} = \begin{bmatrix} -5 & 1 \\ 2x_1 & 0 \end{bmatrix}$$



Scalars: $y, x \in \mathbb{R}$

Vectors: $\mathbf{y} \in \mathbb{R}^d, \mathbf{x} \in \mathbb{R}^c$

Matrices: $\mathbf{X} \in \mathbb{R}^{m \times n}, \mathbf{Y} \in \mathbb{R}^{o \times p}$

Our convention: Numerator dim $\rightarrow$ rows, Denominator dim $\rightarrow$ cols

$\frac{\delta \text{vector}}{\delta \text{matrix}}$

$$\frac{\delta \mathbf{y}}{\delta \mathbf{X}} = \begin{bmatrix} \frac{\delta y_1}{\delta x_{11}} & \frac{\delta y_1}{\delta x_{12}} & \cdots & \frac{\delta y_1}{\delta x_{mn}} \\ \frac{\delta y_2}{\delta x_{11}} & \frac{\delta y_2}{\delta x_{12}} & \cdots & \frac{\delta y_2}{\delta x_{mn}} \\ \vdots & \vdots & \cdots & \vdots \\ \frac{\delta y_d}{\delta x_{11}} & \frac{\delta y_d}{\delta x_{12}} & \cdots & \frac{\delta y_d}{\delta x_{mn}} \end{bmatrix} \quad d \times mn$$

Vectorize the matrix and treat like vector-vector derivative



Scalars: $y, x \in \mathbb{R}$

Vectors: $\mathbf{y} \in \mathbb{R}^d, \mathbf{x} \in \mathbb{R}^c$

Matrices: $\mathbf{X} \in \mathbb{R}^{m \times n}, \mathbf{Y} \in \mathbb{R}^{o \times p}$

Our convention: Numerator dim $\rightarrow$ rows, Denominator dim $\rightarrow$ cols

Chain Rule

$$L(\mathbf{x}) = L(\mathbf{y}(\mathbf{x}))$$

$$\frac{\delta L}{\delta \mathbf{x}} = \frac{\delta L}{\delta \mathbf{y}} \frac{\delta \mathbf{y}}{\delta \mathbf{x}}$$

$1 \times c \qquad 1 \times d \qquad d \times c$



Scalars: $y, x \in \mathbb{R}$

Vectors: $\mathbf{y} \in \mathbb{R}^d, \mathbf{x} \in \mathbb{R}^c$

Matrices: $\mathbf{X} \in \mathbb{R}^{m \times n}, \mathbf{Y} \in \mathbb{R}^{o \times p}$

Our convention: Numerator dim $\rightarrow$ rows, Denominator dim $\rightarrow$ cols

Chain Rule

$$\begin{bmatrix} \frac{\delta L}{\delta x_1} & \frac{\delta L}{\delta x_2} & \cdots & \frac{\delta L}{\delta x_c} \end{bmatrix}_{1 \times c} = \begin{bmatrix} \frac{\delta L}{\delta y_1} & \frac{\delta L}{\delta y_2} & \cdots & \frac{\delta L}{\delta y_d} \end{bmatrix}_{1 \times d} \begin{bmatrix} \frac{\delta y_1}{\delta x_1} & \frac{\delta y_1}{\delta x_2} & \cdots & \frac{\delta y_1}{\delta x_c} \\ \frac{\delta y_2}{\delta x_1} & \frac{\delta y_2}{\delta x_2} & \cdots & \frac{\delta y_2}{\delta x_c} \\ \vdots & \vdots & \cdots & \vdots \\ \frac{\delta y_d}{\delta x_1} & \frac{\delta y_d}{\delta x_2} & \cdots & \frac{\delta y_d}{\delta x_c} \end{bmatrix}_{d \times c}$$



Scalars: $y, x \in \mathbb{R}$

Vectors: $\mathbf{y} \in \mathbb{R}^d, \mathbf{x} \in \mathbb{R}^c$

Matrices: $\mathbf{X} \in \mathbb{R}^{m \times n}, \mathbf{Y} \in \mathbb{R}^{o \times p}$

Our convention: Numerator dim $\rightarrow$ rows, Denominator dim $\rightarrow$ cols

Chain Rule

$$\begin{bmatrix} \frac{\delta L}{\delta x_1} & \frac{\delta L}{\delta x_2} & \cdots & \frac{\delta L}{\delta x_c} \end{bmatrix} = \begin{bmatrix} \frac{\delta L}{\delta y_1} & \frac{\delta L}{\delta y_2} & \cdots & \frac{\delta L}{\delta y_d} \end{bmatrix} \begin{bmatrix} \frac{\delta y_1}{\delta x_1} & \frac{\delta y_1}{\delta x_2} & \cdots & \frac{\delta y_1}{\delta x_c} \\ \frac{\delta y_2}{\delta x_1} & \frac{\delta y_2}{\delta x_2} & \cdots & \frac{\delta y_2}{\delta x_c} \\ \vdots & \vdots & \cdots & \vdots \\ \frac{\delta y_d}{\delta x_1} & \frac{\delta y_d}{\delta x_2} & \cdots & \frac{\delta y_d}{\delta x_c} \end{bmatrix}$$

$$\frac{\delta L}{\delta x_2} = \sum_{i=1}^d \frac{\delta L}{\delta y_i} \frac{\delta y_i}{\delta x_2}$$



Scalars: $y, x \in \mathbb{R}$

Vectors: $\mathbf{y} \in \mathbb{R}^d, \mathbf{x} \in \mathbb{R}^c$

Matrices: $\mathbf{X} \in \mathbb{R}^{m \times n}, \mathbf{Y} \in \mathbb{R}^{o \times p}$

$\frac{\delta^2 \text{scalar}}{\delta \text{vector}^2}$

$$\frac{\delta^2 y}{\delta \mathbf{x}^2} = \begin{bmatrix} \frac{\delta^2 y}{\delta x_1^2} & \frac{\delta^2 y}{\delta x_1 x_2} & \cdots & \frac{\delta^2 y}{\delta x_1 x_c} \\ \frac{\delta^2 y}{\delta x_2 x_1} & \frac{\delta^2 y}{\delta x_2^2} & \cdots & \frac{\delta^2 y}{\delta x_2 x_c} \\ \vdots & \vdots & \cdots & \vdots \\ \frac{\delta^2 y}{\delta x_c x_1} & \frac{\delta^2 y}{\delta x_c x_2} & \cdots & \frac{\delta^2 y}{\delta x_c^2} \end{bmatrix}_{c \times c}$$

**Called the Hessian
Is symmetric.**



Scalars: $y, x \in \mathbb{R}$

Vectors: $\mathbf{y} \in \mathbb{R}^d, \mathbf{x} \in \mathbb{R}^c$

Matrices: $\mathbf{X} \in \mathbb{R}^{m \times n}, \mathbf{Y} \in \mathbb{R}^{o \times p}$

$\frac{\delta^2 \text{vector}}{\delta \text{vector}^2}$

$$\frac{\delta^2 \mathbf{y}}{\delta \mathbf{x}^2} = \begin{bmatrix} \frac{\delta^2 y_1}{\delta x_1^2} & \frac{\delta^2 y_1}{\delta x_1 x_2} & \cdots & \frac{\delta^2 y_1}{\delta x_1 x_c} \\ \frac{\delta^2 y_1}{\delta x_2 x_1} & \frac{\delta^2 y_1}{\delta x_2^2} & \cdots & \frac{\delta^2 y_1}{\delta x_2 x_c} \\ \vdots & \vdots & \cdots & \vdots \\ \frac{\delta^2 y_1}{\delta x_c x_1} & \frac{\delta^2 y_1}{\delta x_c x_2} & \cdots & \frac{\delta^2 y_1}{\delta x_c^2} \end{bmatrix} \cdots \begin{bmatrix} \frac{\delta^2 y_d}{\delta x_1^2} & \frac{\delta^2 y_d}{\delta x_1 x_2} & \cdots & \frac{\delta^2 y_d}{\delta x_1 x_c} \\ \frac{\delta^2 y_d}{\delta x_2 x_1} & \frac{\delta^2 y_d}{\delta x_2^2} & \cdots & \frac{\delta^2 y_d}{\delta x_2 x_c} \\ \vdots & \vdots & \cdots & \vdots \\ \frac{\delta^2 y_d}{\delta x_c x_1} & \frac{\delta^2 y_d}{\delta x_c x_2} & \cdots & \frac{\delta^2 y_d}{\delta x_c^2} \end{bmatrix}_{d \times c \times c}$$

Hessian is a tensor for vector-vector functions.



Resources for Vector Calc:

- The Matrix Cookbook - www.math.uwaterloo.ca/~hwolkowi/matrixcookbook.pdf

2	Derivatives	8
2.1	Derivatives of a Determinant	8
2.2	Derivatives of an Inverse	9
2.3	Derivatives of Eigenvalues	10
2.4	Derivatives of Matrices, Vectors and Scalar Forms	10
2.5	Derivatives of Traces	12
2.6	Derivatives of vector norms	14
2.7	Derivatives of matrix norms	14
2.8	Derivatives of Structured Matrices	14

- Any multivariable calculus book for fundamentals

Warning: be careful with notation conventions for rows vs. columns, varies by author.



Advice: When in doubt, just write out the equation in scalars for simple examples.

$$\nabla_x x^T A x = ?$$



Advice: When in doubt, just write out the equation in scalars for simple examples.

$$x^T A x = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = x_1(x_1 a_{11} + x_2 a_{21}) + x_2(x_1 a_{12} + x_2 a_{22})$$



$$\frac{\partial x^T A x}{\partial x_1} = 2x_1 a_{11} + x_2 a_{21} + x_2 a_{12}$$

$$\frac{\partial x^T A x}{\partial x_2} = x_1 a_{21} + x_1 a_{12} + 2x_2 a_{22}$$



$$\nabla_x x^T A x = x^T (A + A^T)$$

$$\begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 2a_{11} & a_{12} + a_{21} \\ a_{21} + a_{12} & 2a_{22} \end{bmatrix} = \begin{bmatrix} 2x_1 a_{11} + x_2 a_{21} + x_2 a_{12} \\ x_1 a_{21} + x_1 a_{12} + 2x_2 a_{22} \end{bmatrix}$$

$2x^T A$ for
symmetric A

Today's Learning Objectives



Be able to answer:

- ~~What is a loss function?~~
 - ~~What are typical loss functions?~~
- ~~What are gradients? Jacobians? Hessians?~~
- What is optimization and why do we care?
 - What are first-order methods?
 - What is gradient descent?
 - When does gradient descent reach optima?
 - What makes a function convex / non-convex?
 - What is the intuition behind second order methods?
 - Why are they typically not used in deep learning?



Someone hands you an equation $f(\mathbf{x})$ and asks – “What value of $\mathbf{x}$ would make $f(\mathbf{x})$ smallest?” – now what?

Is the domain of f discrete or continuous?

Is f differentiable? Twice differentiable?

Is f convex or non-convex?

Are there any constraints on $\mathbf{x}$?

Warning: Optimization is a whole field you could dedicate your life’s work to. We are barely touching the surface – specifically looking at a couple of relevant techniques in gradient-based optimization.

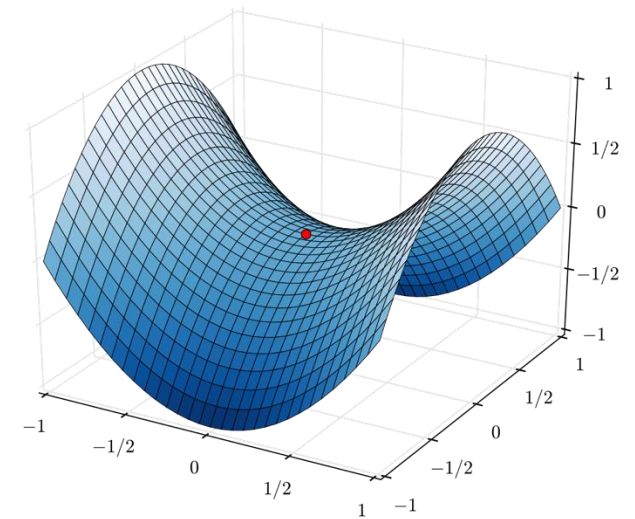
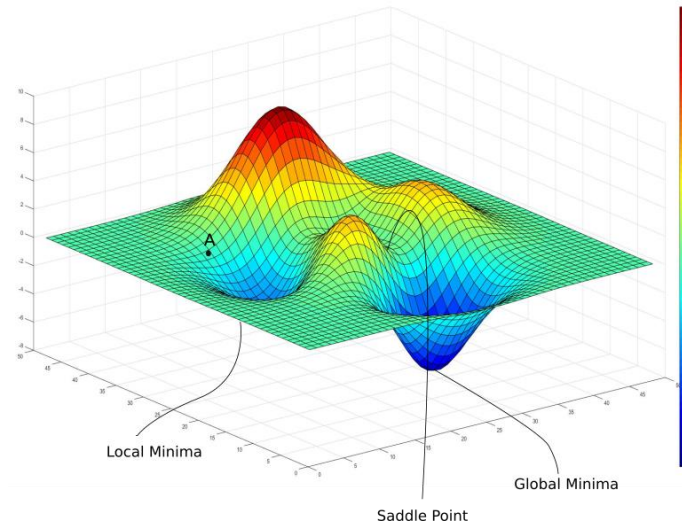
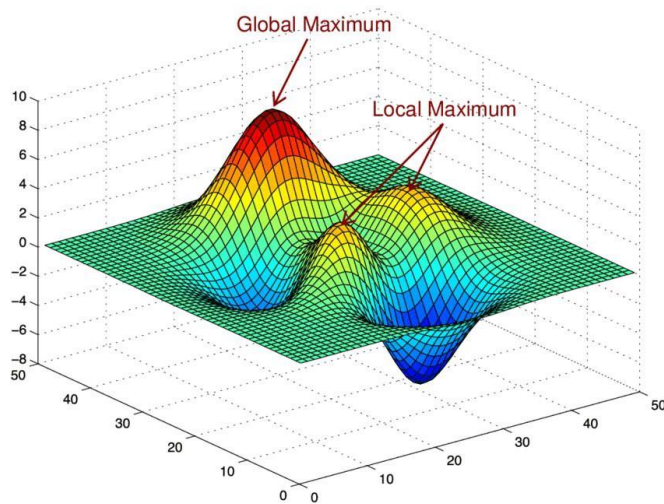


Someone hands you a differentiable $f(\mathbf{x})$ and asks for $w^* = \min_w f(\mathbf{x})$?

Your calculus class would say something like:

- 1) Find critical points of the function where the gradient is all zeros

$$\nabla_w f(\mathbf{x}) = \mathbf{0}$$





Someone hands you a differentiable $f(\mathbf{x})$ and asks for $w^* = \min_w f(\mathbf{x})$?

Your calculus class would say something like:

1) Find critical points of the function where the gradient is all zeros

$$\nabla_w f(\mathbf{x}) = \mathbf{0}$$

2) Test the eigenvalues of the Hessian at each critical point $\mathbf{x}_0$:

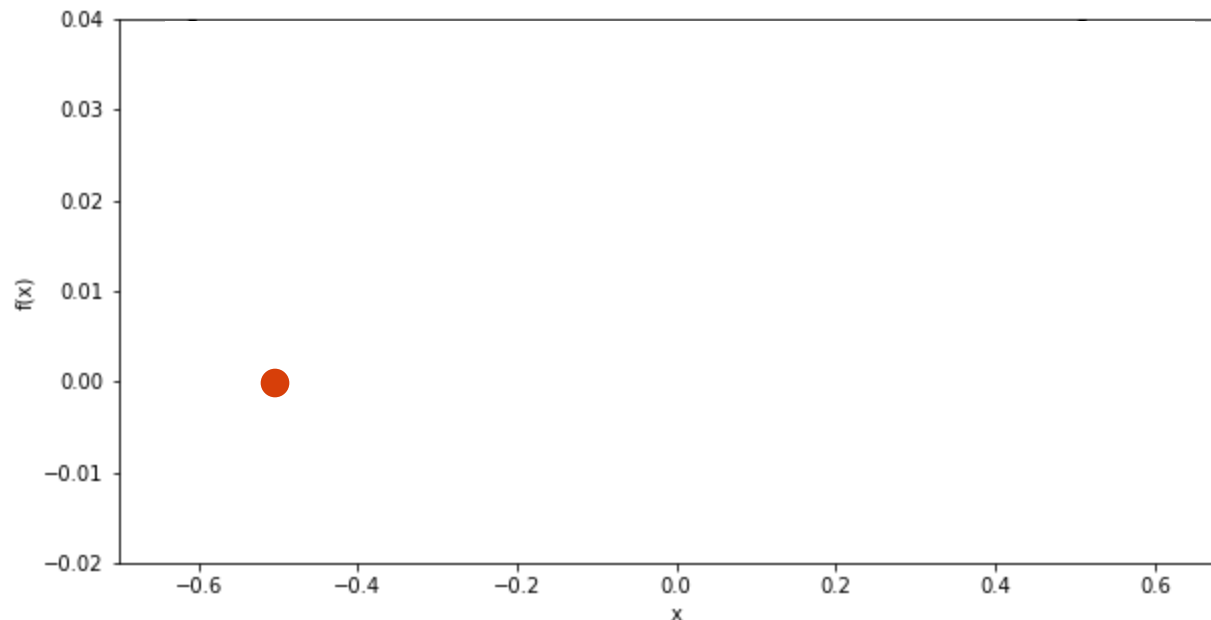
- If some eigenvalues > 0 and some eigenvalues < 0 , $\mathbf{x}_0$ is a saddle point
- If all eigenvalues ≥ 0 , $\mathbf{x}_0$ is (at least) a local minimum
- If all eigenvalues ≤ 0 , $\mathbf{x}_0$ is (at least) a local maximum



$$f(x) = (x - 0.2)(x + 0.5)(x - 0.3)(x + 0.1)$$

(Note: This is analytically solvable,
but play along for now)

Okay - fine. So you plug in some value for the function: $f(-0.5) = 0$



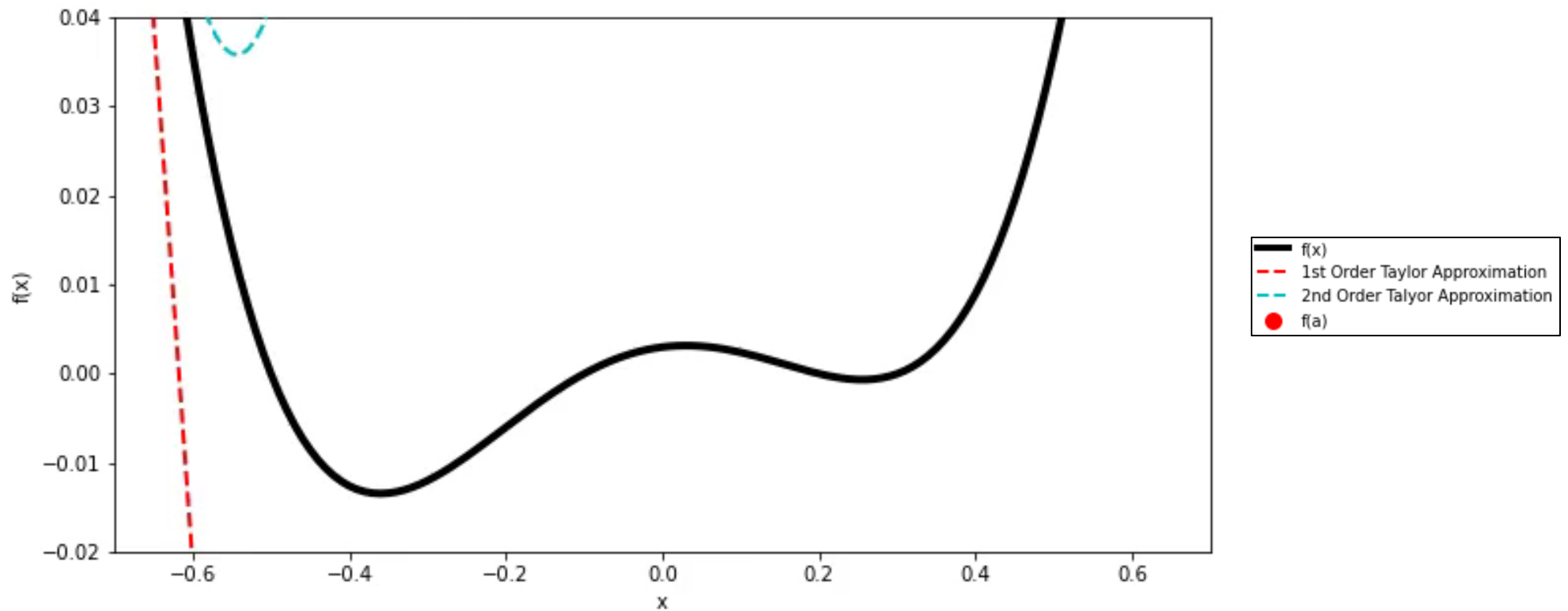
Can we move from here to find a minimum? Should we go left or right? How far?

One idea: Make a local approximation of the function and move to minimize that.



Taylor Series Approximation:

$$f(x) \approx f(a) + \nabla f(a)^T(x - a) + \frac{1}{2}(x - a)^T H_f(a)(x - a) + \dots$$



1st Order Taylor Series Approximation: $f(x) \approx f(a) + \nabla f(a)^T(x - a)$

Consider moving from a to a new parameter value $a + \Delta x$, what choice of Δx minimizes $f(x)$?

Our Taylor approximation suggests: $f(a + \Delta x) \approx f(a) + \nabla f(a)^T \Delta x$

$$\min_{\Delta x} f(a) + \underbrace{\nabla f(a)^T \Delta x}_{\Delta x = -\alpha \nabla f(a)}$$

v

w

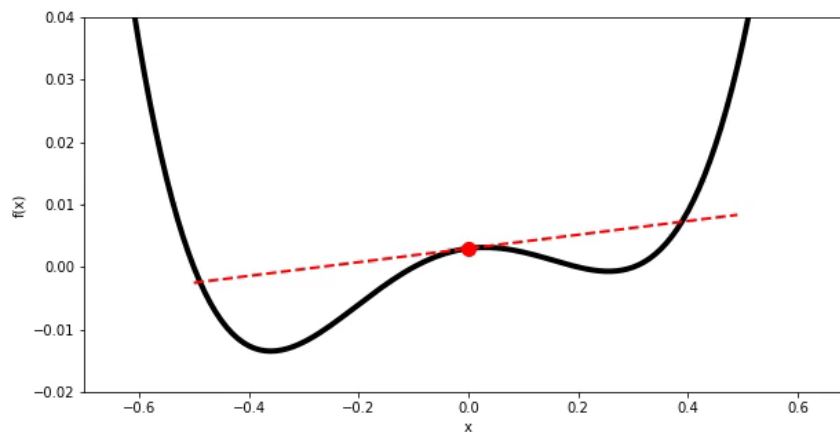
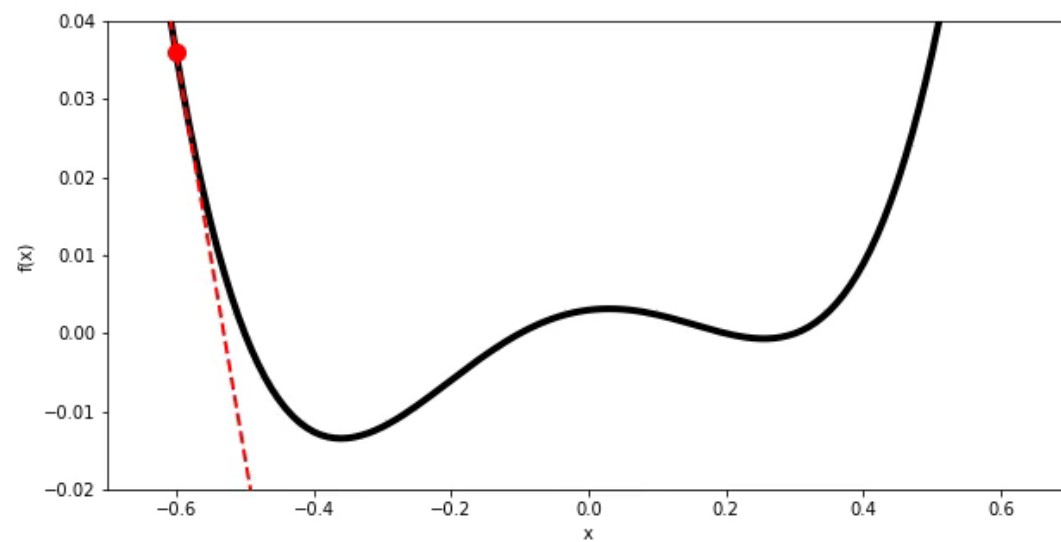
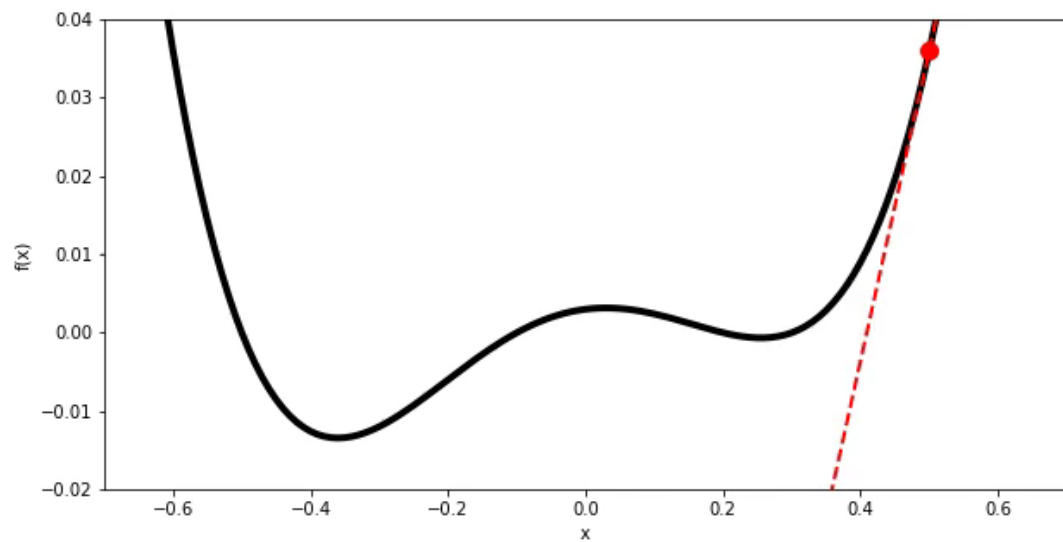
θ

$$v^T w = |v||w| \cos \theta$$

What value for α minimizes $f(a - \alpha \nabla f(a))$?

But wait! Our Taylor series approximation is only valid near a ! Deciding on the step-size α is tricky business.

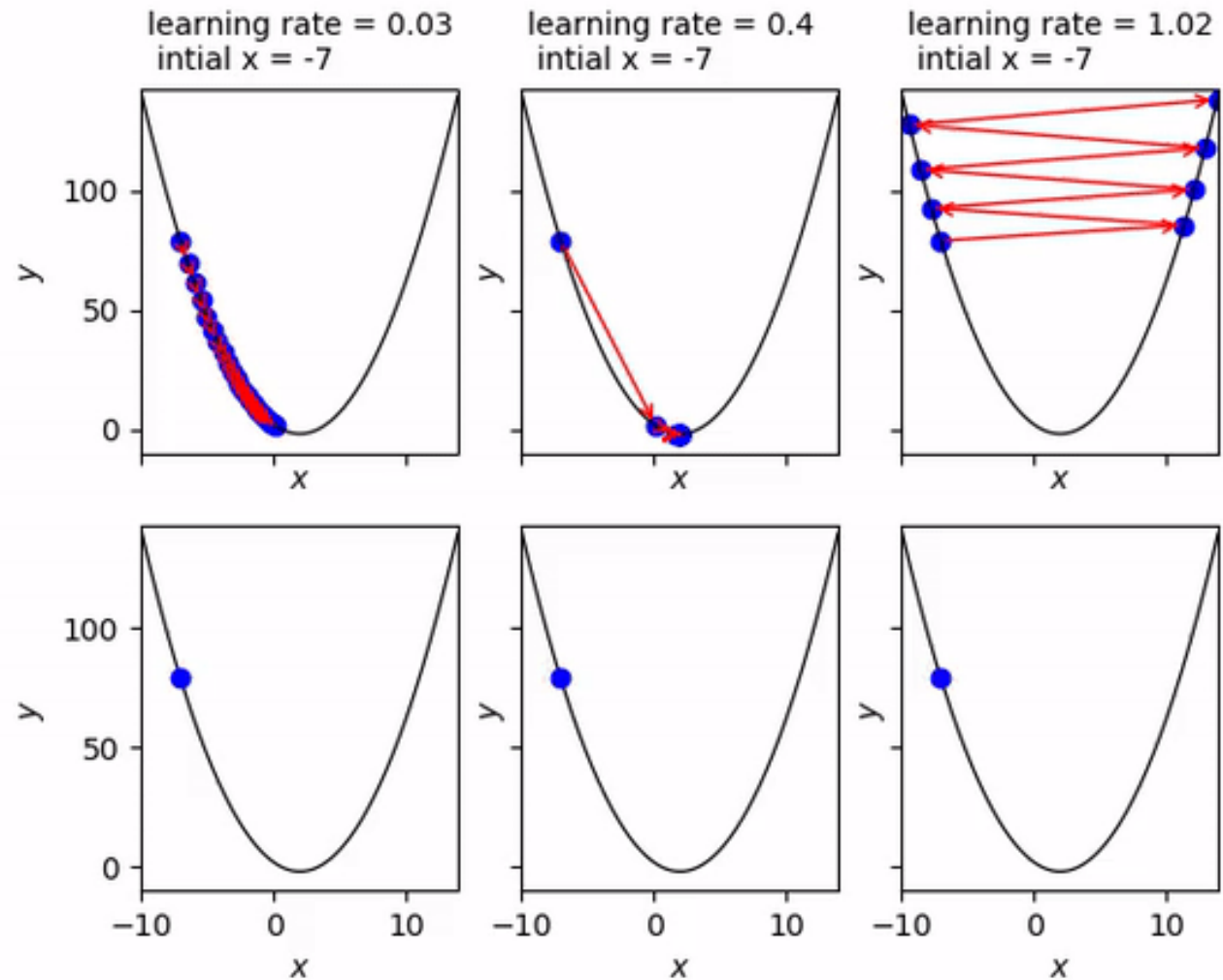
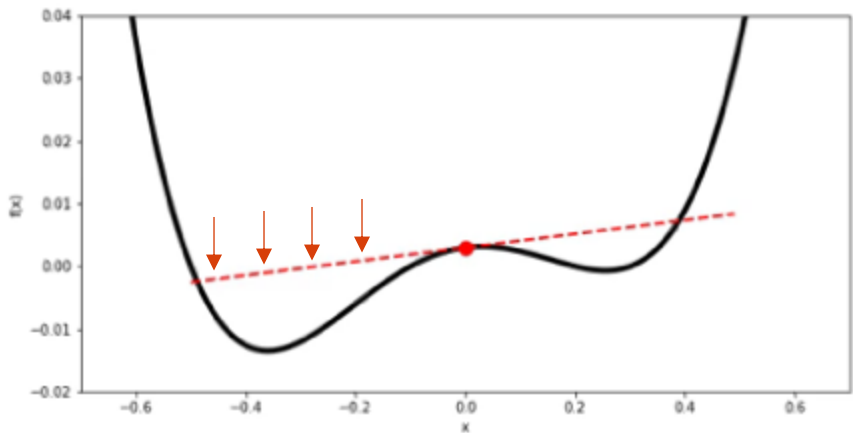
Gradient descent update rule: $x^{(t+1)} = x^{(t)} - \alpha \nabla f(x^{(t)})$ $\alpha > 0$



Setting α appropriately matters

- Called step-size or learning rate
- Deep learning typically decays learning rate over times

Optimization methods sometimes do line search to find good α per step:



Using higher-order approximations can remove the need for this.

2nd Order Taylor Series Approximation: $f(x) \approx f(a) + \nabla f(a)^T(x - a) + \frac{1}{2}(x - a)^T H_f(a)(x - a)$

Consider moving from a to a new parameter value $a + \Delta x$, what choice of Δx minimizes $f(x)$?

Our Taylor approximation suggests: $f(a + \Delta x) \approx f(a) + \nabla f(a)^T \Delta x + \frac{1}{2} \Delta x^T H_f(a) \Delta x$

$$\min_{\Delta x} f(a) + \nabla f(a)^T \Delta x + \frac{1}{2} \Delta x^T H_f(a) \Delta x$$

$$\nabla_{\Delta x} \left(f(a) + \nabla f(a)^T \Delta x + \frac{1}{2} \Delta x^T H_f(a) \Delta x \right) = \nabla f(a) + \Delta x^T H_f(a)$$

$$\Rightarrow \Delta x = -H_f(a)^{-1} \nabla f(a)$$



Inverse of Hessian

Reminder:

Notation for Vector Calculus

Advice: When in doubt, just write out the equation in scalars for simple examples.

$$x^T A x = [x_1 \quad x_2] \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = x_1(x_1 a_{11} + x_2 a_{21}) + x_2(x_1 a_{12} + x_2 a_{22})$$

$$\frac{\partial x^T A x}{\partial x_1} = 2x_1 a_{11} + x_2 a_{21} + x_2 a_{12} \quad \frac{\partial x^T A x}{\partial x_2} = x_1 a_{21} + x_1 a_{12} + 2x_2 a_{22}$$

$$\nabla_x x^T A x = x^T (A + A^T)$$

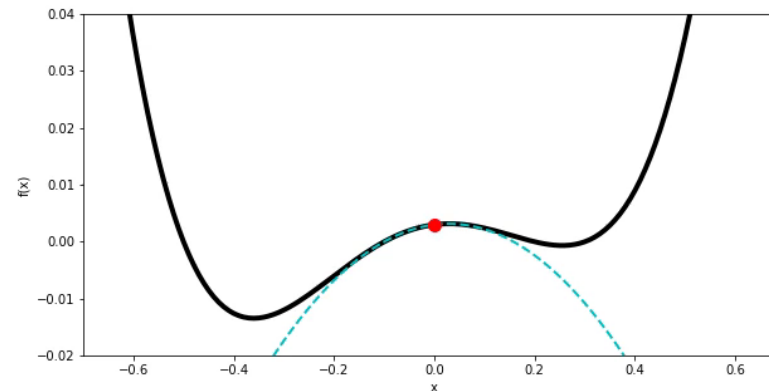
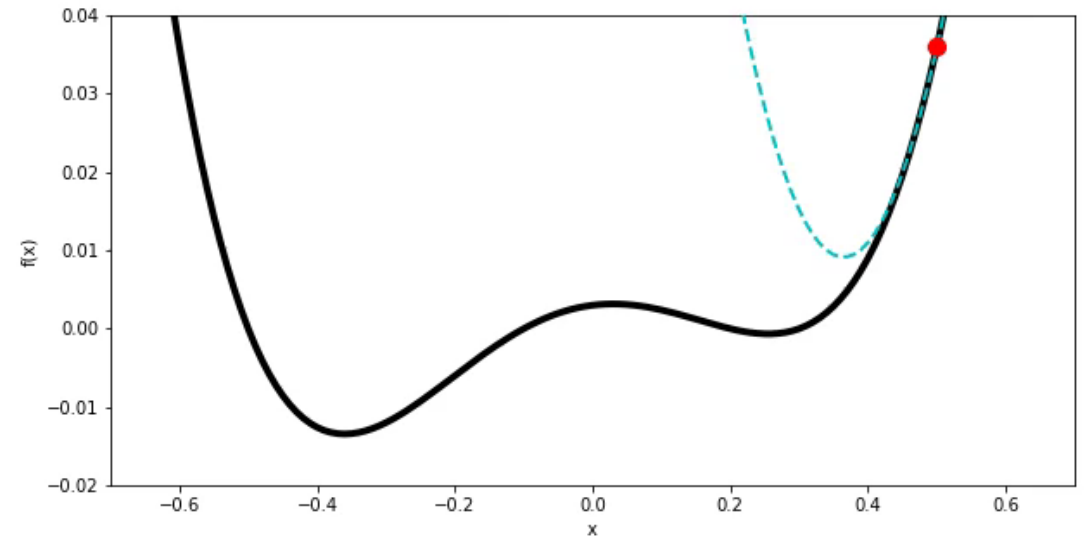
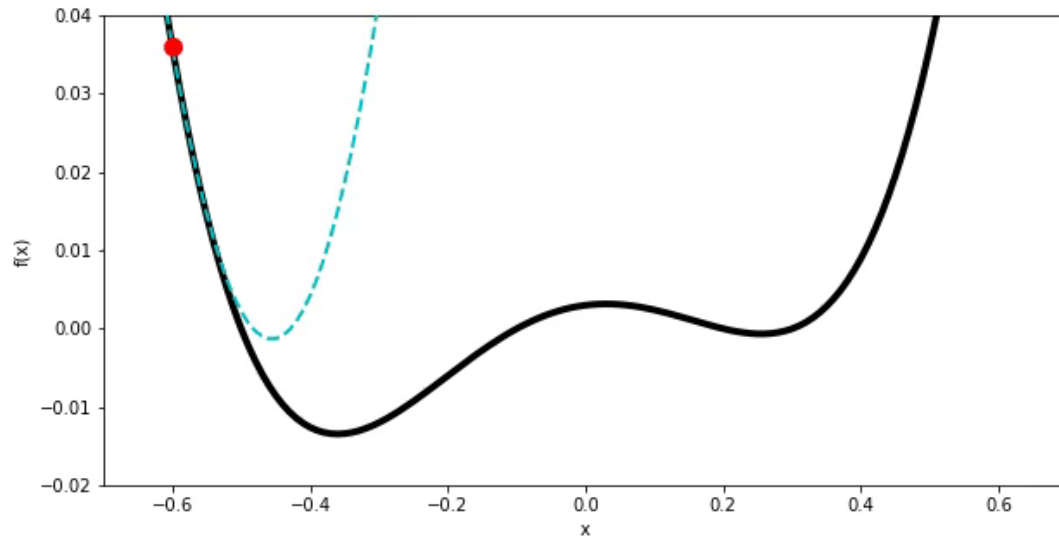
$2x^T A$ for symmetric A

$$\begin{bmatrix} \frac{\delta^2 y}{\delta x_c \delta x_1} & \frac{\delta^2 y}{\delta x_c \delta x_2} & \dots & \frac{\delta^2 y}{\delta x_c^2} \end{bmatrix} c \times c$$

Called the Hessian
Is symmetric.

Newton Method update rule:

$$x^{t+1} = x^t - H_f(x^t)^{-1} \nabla f(x^t)$$



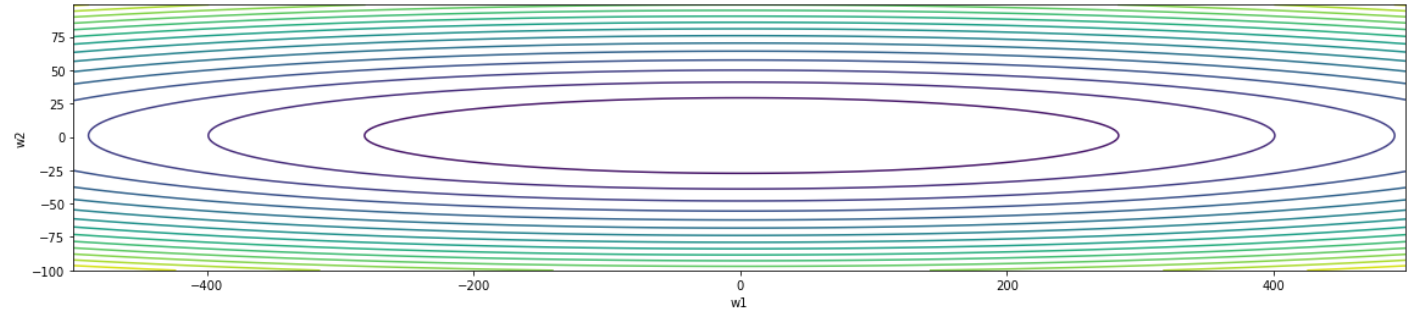
Newton method finds roots of the gradient – not necessarily minima





A Numerical Example

$$f(\mathbf{w}) = (w_1 - 1)^2 + 100(w_2 - 1)^2$$



What value of $\mathbf{w}$ minimizes this and value does it take?

What is gradient $\nabla_{\mathbf{w}} f$ of this function?

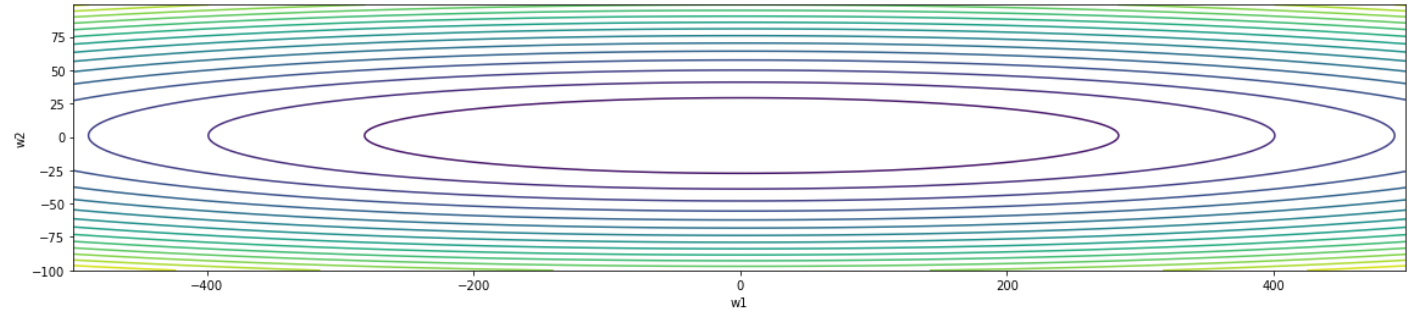
What is $\nabla_{\mathbf{w}} f$ at $\mathbf{w} = [0, 0]^T$?

For gradient descent, what would a good step size be?



A Numerical Example

$$f(\mathbf{w}) = (w_1 - 1)^2 + 100(w_2 - 1)^2$$



What is gradient $\nabla_{\mathbf{w}} f$ of this function?

$$[2(w_1 - 1), 200(w_2 - 1)]^T$$

What is $\nabla_{\mathbf{w}} f$ at $\mathbf{w} = [0, 0]^T$?

$$[-2, -200]^T$$

What is the Hessian $\nabla_{\mathbf{w}}^2 f$ (aka H_f)?

$$\begin{bmatrix} 2 & 0 \\ 0 & 200 \end{bmatrix}$$

What is the result of a single Newton Method step?

$$\begin{bmatrix} 0 \\ 0 \end{bmatrix} - \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{200} \end{bmatrix} \begin{bmatrix} -2 \\ -200 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$



f is convex if any of these three conditions are met:

Secant line higher than function over interval:

$$f(tx_1 + (1 - t)x_2) \leq tf(x_1) + (1 - t)f(x_2)$$

$$\forall x_1, x_2 \in X, \quad \forall t \in [0,1]$$

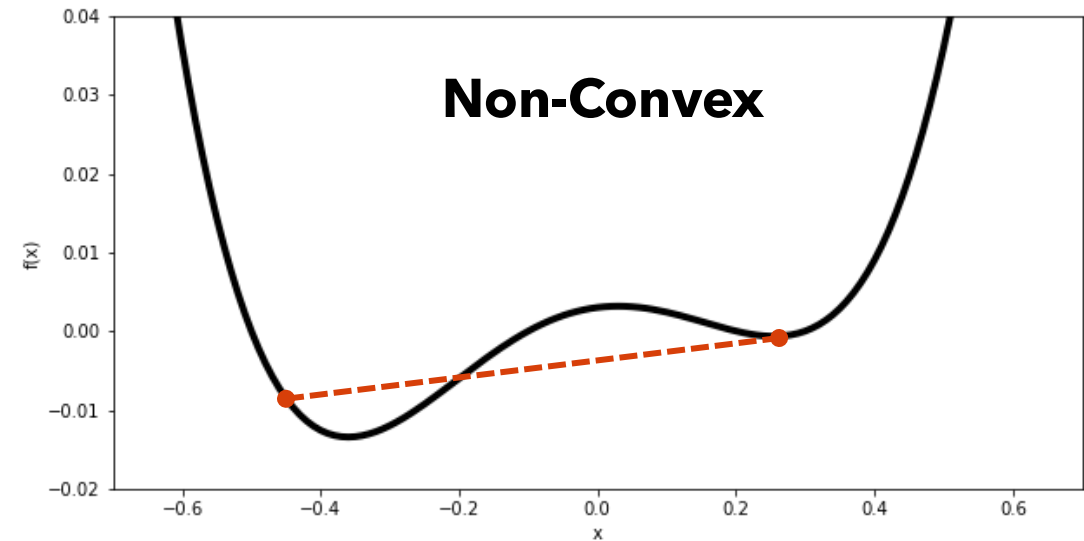
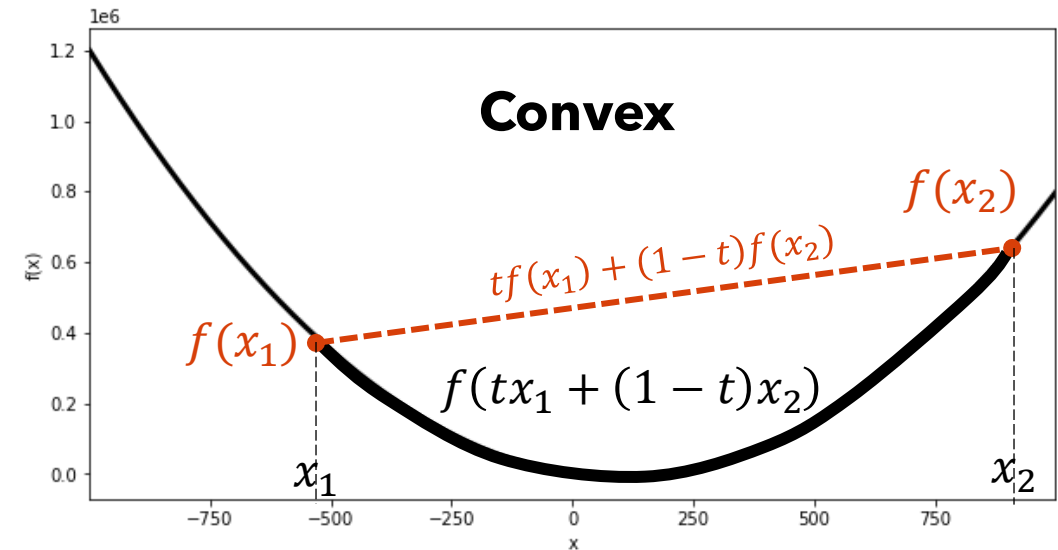
1st Order:

$$f(x_2) \geq f(x_1) + \nabla f(x_1)^T (x_2 - x_1)$$

$$\forall x_1, x_2 \in X$$

2nd Order: Positive-Semidefinite Hessian

$$\nabla^2 f(x) \geq 0$$





Do you need to compute the Hessian for Newton methods?

For regular Newton methods, yes

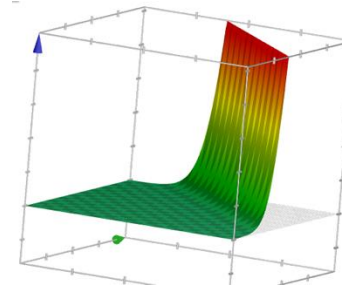
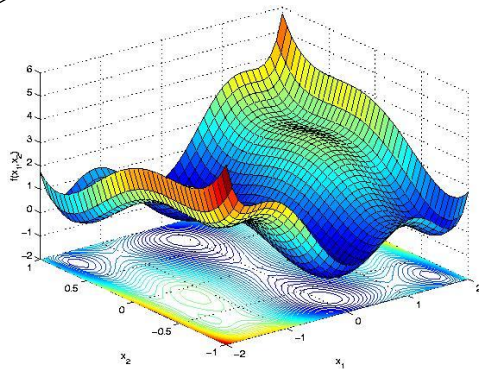
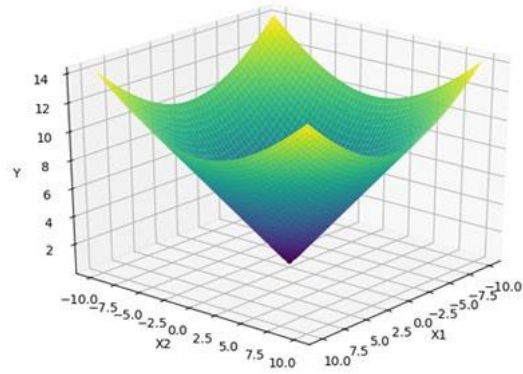
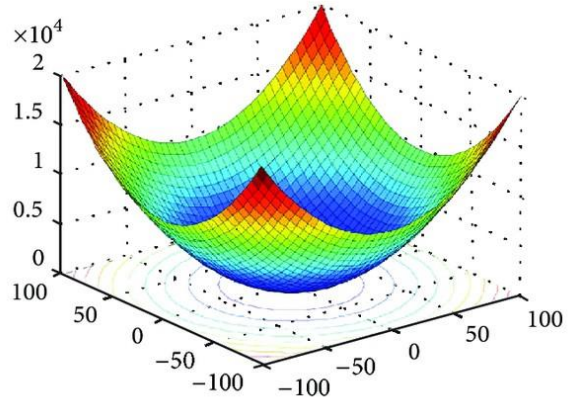
But there are many “cheaper” second-order methods such as limited-memory versions (using a few vectors to approximate the Hessian eigenvectors, e.g. LBFGS)

Some of these are making their ways into deep learning (e.g. Shampoo, Sophia), but they are quite advanced approaches and the exact comparisons between them and first-order methods are not completely clear yet

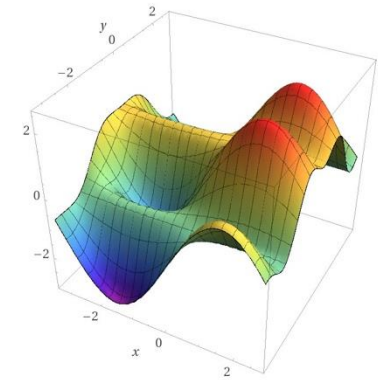
For this class, we will stick with first-order methods, but please always keep an eye on second-order ones and make sure you understand that optimization of deep learning is still a super-interesting problem that has a lot of room for improvement



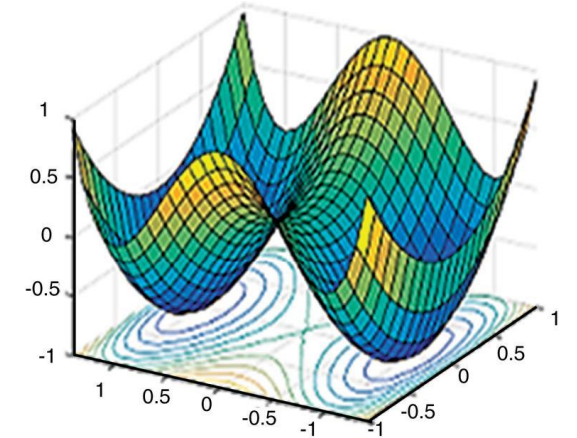
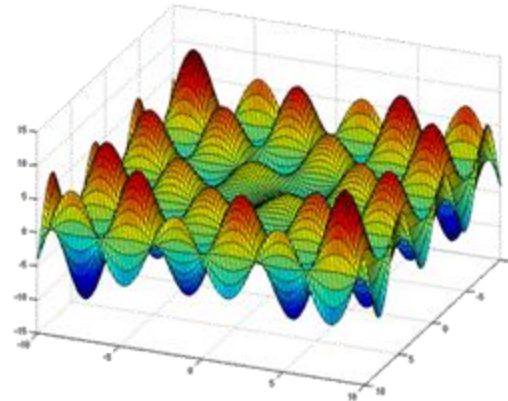
Convex



Non-Convex



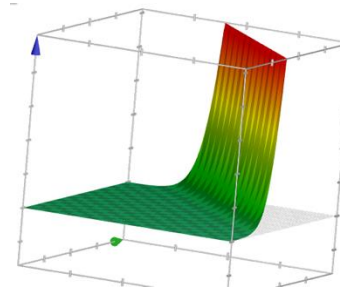
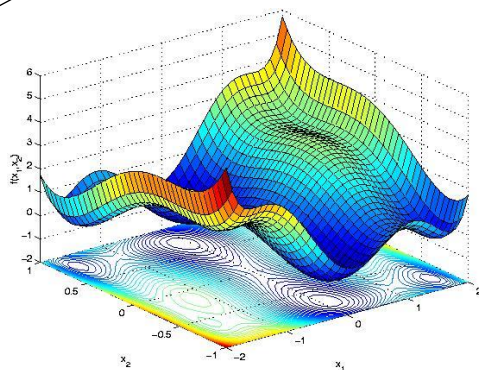
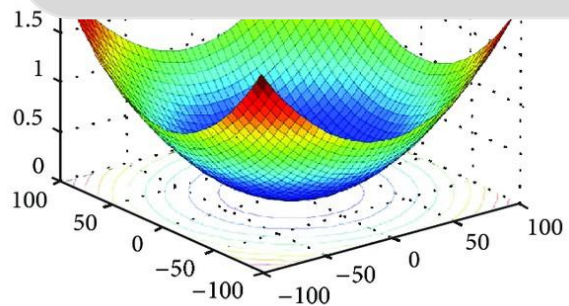
Computed by Wolfram|Alpha



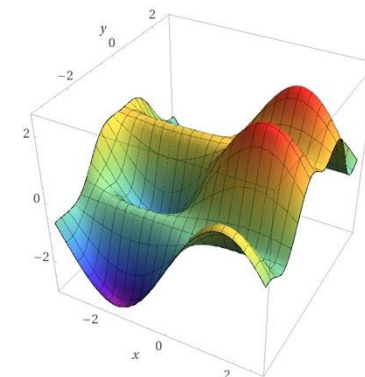


Convex Optimization

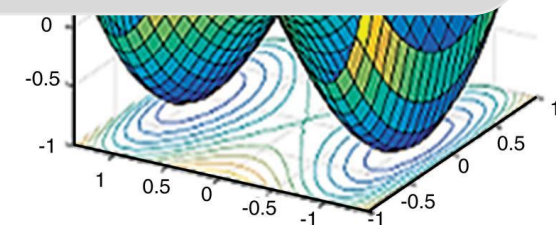
Strong guarantees about global optimality
under reasonable assumptions



Non-Convex Optimization



No promises.



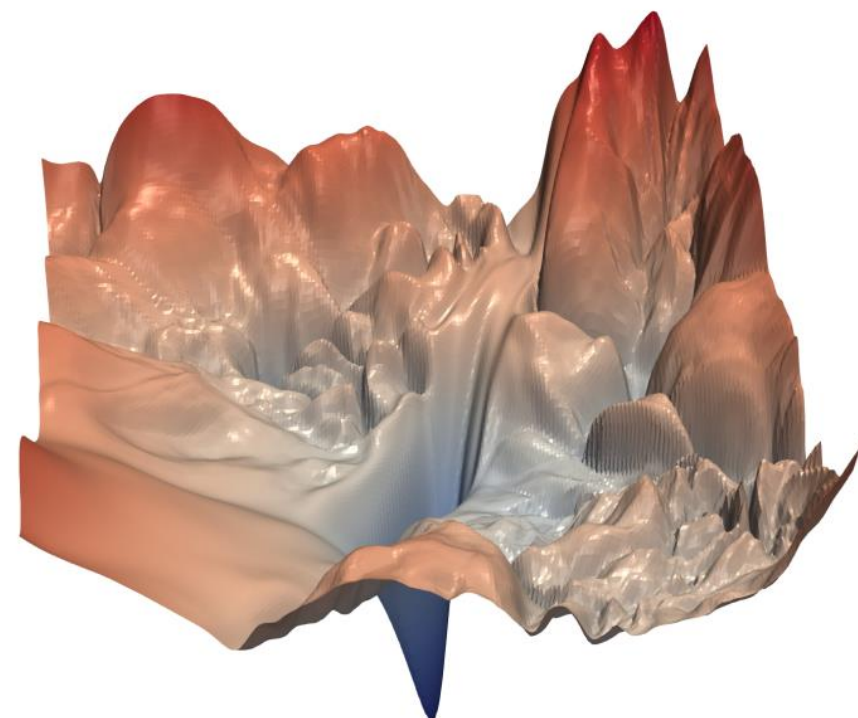


Neural Networks are Non-Convex (Often extremely so)

Neural networks are highly non-convex, but we are going to be using simple algorithms like gradient descent.

Warning: Here there be dragons.

- Extremely unlikely to reach global optima in loss
- Your algorithms may not work off-the-shelf
 - You will need to tune learning rates
- Lots of add-ons to make gradient descent better
- Initialization will matter greatly
 - Conventional wisdom about good/bad ones
 - Limited (but growing) theory
- Model design will impact optimization characteristics
 - See soon – ReLU and residual connections



2D visualization of ResNet-50 loss surface without residual connections.
Not actual surface.

Today's Learning Objectives

Be able to answer:

- ~~What is a loss function?~~
 - ~~What are typical loss functions?~~
- ~~What are gradients? Jacobians? Hessians?~~
- ~~What is optimization and why do we care?~~
 - ~~What are first-order methods?~~
 - ~~What is gradient descent?~~
 - ~~When does gradient descent reach optima?~~
 - ~~What makes a function convex / non-convex?~~
 - ~~What is the intuition behind second order methods?~~
 - ~~Why are they typically not used in deep learning?~~

